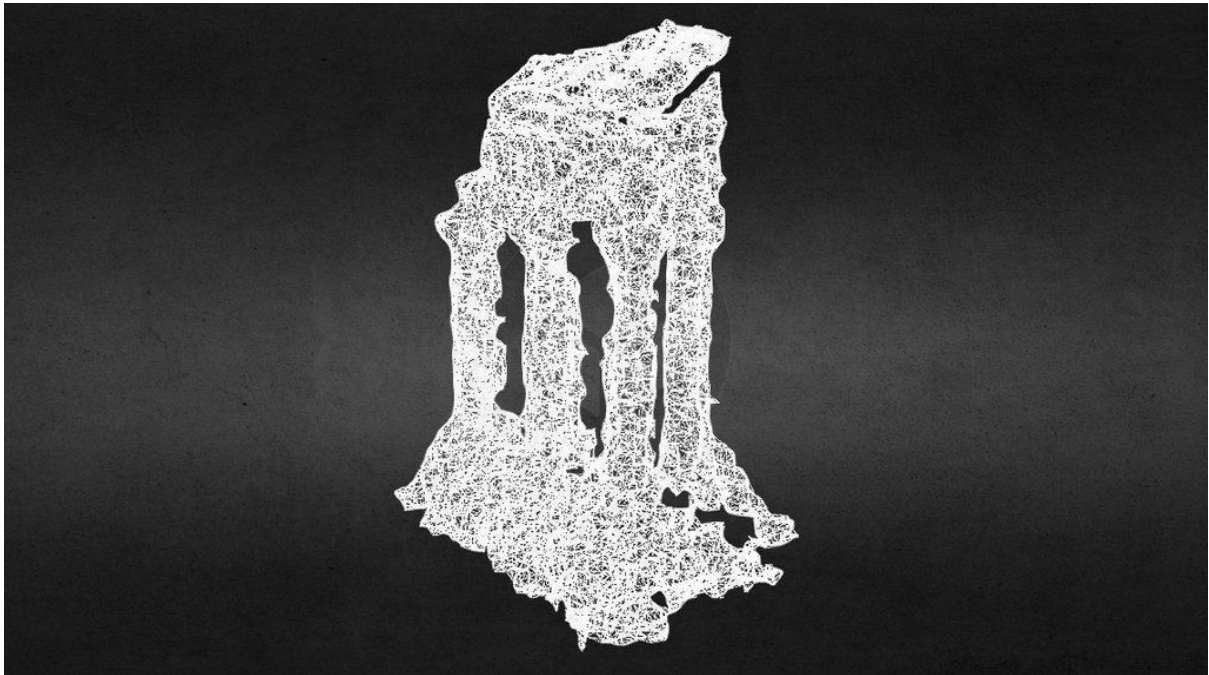




3D Geometry Reconstruction Software

Group Members

Qadeem Khan
Syed Sameed Ahmed
Arham Sajjad



Submitted To

Professor Yohan Fourgerolle

Computer Vision and Robotics

Université Bourgogne Europe
May 2, 2025

Contents

1	Introduction	3
2	Problem Statement	3
3	Methodology	4
4	Camera calibration	4
4.1	Calibration Input Data and Setup	4
4.2	Chessboard Corner Detection	4
4.3	Visualization of Detected Corners	5
4.4	Intrinsic Parameters,Distortion Coefficients and Reprojection Error	5
4.4.1	Stereo Setup from Single Camera	5
4.4.2	Stereo Setup from Two Cameras	6
4.5	Challenges	7
5	Image Undistortion	7
6	Image Rectification	8
6.1	Single Camera Setup	8
6.2	Dual-Camera Setup	9
6.3	Challenges	9
7	Sparse 3D Reconstruction	9
7.1	Feature Matching	9
7.1.1	Single Camera Setup	9
7.1.2	Dual Camera Setup	10
7.2	Challenges	10
7.3	Finding Fundamental Matrix	11
7.3.1	Single Camera Setup	11
7.3.2	Dual Camera Setup	11
7.4	Extracting Essential Matrix and Camera Pose	12
7.4.1	Single Camera Setup	12
7.4.2	Dual Camera Setup	13
7.5	Triangulation	14
7.5.1	Single Camera Setup	14
7.5.2	Dual Camera Setup	15
8	Depth MAP from Stereo Images	16
8.1	The Semi-Global Block Matching (SGBM)	16
8.2	Disparity Map Computation	16
8.3	Challenges	18
9	Graphical User Interface	18
9.1	Calibration	18
9.2	Rectification	19
9.3	3D Reconstruction	20
9.3.1	Sparse	20
9.3.2	Dense	22
10	Future Improvements	23
11	Conclusion	23
12	References	24

List of Figures

1	Stereo 3D Reconstruction Pipeline	4
2	Detected Corner Points	5
3	undistorted (left) and (right) images	8
4	Rectification result for a single camera.	8
5	Rectification result for a Dual camera.	9
6	Feature Matching from single camera approach.	10
7	Feature Matching from dual camera approach.	10
8	Reprojected 3D points onto the original image	15
9	Reprojected 3D points onto the original image	15
10	Disparity map	17
11	Disparity map	17
12	GUI home page	18
13	camera parameters attained through calibration	19
14	Rectified left and right image	19
15	Selection for 3D reconstruction	20
16	Matched features in rectified images	21
17	Reprojection of the points onto the image	21
18	Log showing parameters attained through triangulation	22
19	Disparity Map	22

1 Introduction

Stereo vision is a fundamental technique in computer vision, enabling depth perception and 3D reconstruction from two images captured by a pair of cameras. This project aims to develop a portable and self-contained software solution for 3D geometry reconstruction using two USB cameras connected to a standard computer. The system will process stereo image pairs to generate a 3D point cloud, providing a practical demonstration of depth estimation and spatial reconstruction.

The project is structured into distinct, modular components to ensure a systematic approach to development and evaluation. These components include camera calibration, stereo rectification, feature detection and matching, stereo geometry estimation, triangulation, and point cloud post-processing. Each module is designed to be independently implemented and assessed, ensuring clarity and robustness in the overall system.

2 Problem Statement

The objective of this project is to explore and implement two approaches for stereo-based 3D reconstruction:

- Simulating a stereo vision setup using a single moving camera, capturing two images at a known baseline offset.
- Performing stereo reconstruction using a two-camera system setup

For both approaches the solution addresses the following key technical challenges:

1. Extracting intrinsic parameters, lens distortions and relative position and orientation from the camera calibration.
2. Performing rectification to align the images from both cameras so that corresponding epipolar lines are horizontal, simplifying feature matching.
3. Identify and match key features (e.g., corners, edges) between the two images to establish correspondences for 3D reconstruction.
4. Compute the essential matrix and derive the relative camera poses (rotation and translation) to model the geometric relationship between the cameras.
5. Use matched features and camera poses to triangulate 3D points, generating a sparse point cloud representing the scene geometry.
6. Implement a stereo matching algorithm (e.g., Semi-Global Block Matching, SGBM) to compute a disparity map from the rectified image pairs
7. Refine the point cloud by removing noise, smoothing data, and enhancing overall quality for practical use.

3 Methodology

Following pipeline was followed to get 3D point cloud and depth map from the 2D images captured from the stereo system.

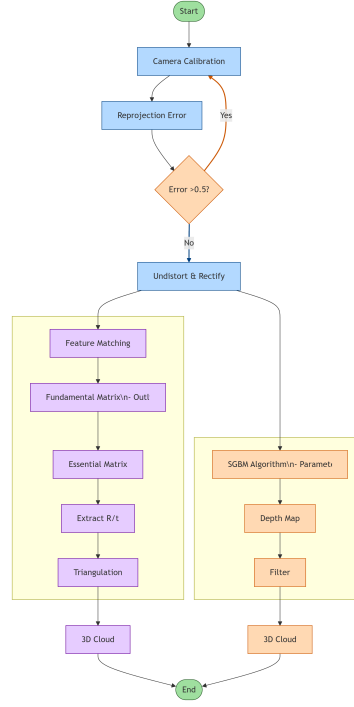


Figure 1: Stereo 3D Reconstruction Pipeline

4 Camera calibration

Camera calibration process for both single and stereo camera setups using a series of images containing a chessboard pattern. For single camera calibration, the objective is to determine the intrinsic parameters—such as focal length, optical center, and distortion coefficients—which are essential for accurate measurements and computer vision tasks like 3D reconstruction. This involves detecting chessboard corners in multiple images and mapping the 2D image coordinates to corresponding 3D real-world points. In this setup, 8 calibration images were used for the single camera. For stereo calibration, the process is extended to two cameras (left and right), where each camera is first calibrated independently to estimate its intrinsic parameters. A total of 150 images were captured for the stereo setup. These calibrations are performed using OpenCV’s `calibrateCamera()` function and validated through reprojection error analysis. The resulting parameters form the basis for stereo calibration. We used `stereoCalibrate()` function of OpenCV, where intrinsic parameters and distortion coefficients are passed in this function and it returns the relative rotation and translation matrix with fundamental and essential matrix.

4.1 Calibration Input Data and Setup

Each image contained a chessboard pattern with 7 columns and 9 rows of inner corners. The size of each square in the chessboard is 15mm. The process began by loading each image using OpenCV’s `cv2.imread()` function. If an image failed to load, an error message was logged, and the process moved to the next image. Once loaded, the images were converted to grayscale using `cv2.cvtColor()` to simplify the corner detection process.

4.2 Chessboard Corner Detection

The corners of the chessboard were detected using OpenCV’s `cv2.findChessboardCorners()` function. This function takes the grayscale image and the chessboard size as inputs and returns a boolean value (`ret`) indicating whether the corners were successfully detected, along with the pixel coordinates of the

detected corners (corners). If corners were detected, the 3D real-world coordinates of the chessboard corners (objp) and the 2D image points (corners) were stored in the lists objpoints and imgpoints, respectively. These lists were used to accumulate data from all calibration images for subsequent camera calibration.

4.3 Visualization of Detected Corners

To visualize, the detected corners were drawn on the original images using `cv2.drawChessboardCorners()`. The images were then displayed using Matplotlib (`plt.imshow()`) with the detected corners highlighted. This step allowed for visual confirmation of the accuracy of the corner detection process. If the chessboard corners were not detected in an image, a message was logged, and the process continued to the next image. In this case, all calibration images successfully yielded detected corners, and no errors were encountered during image loading or processing.

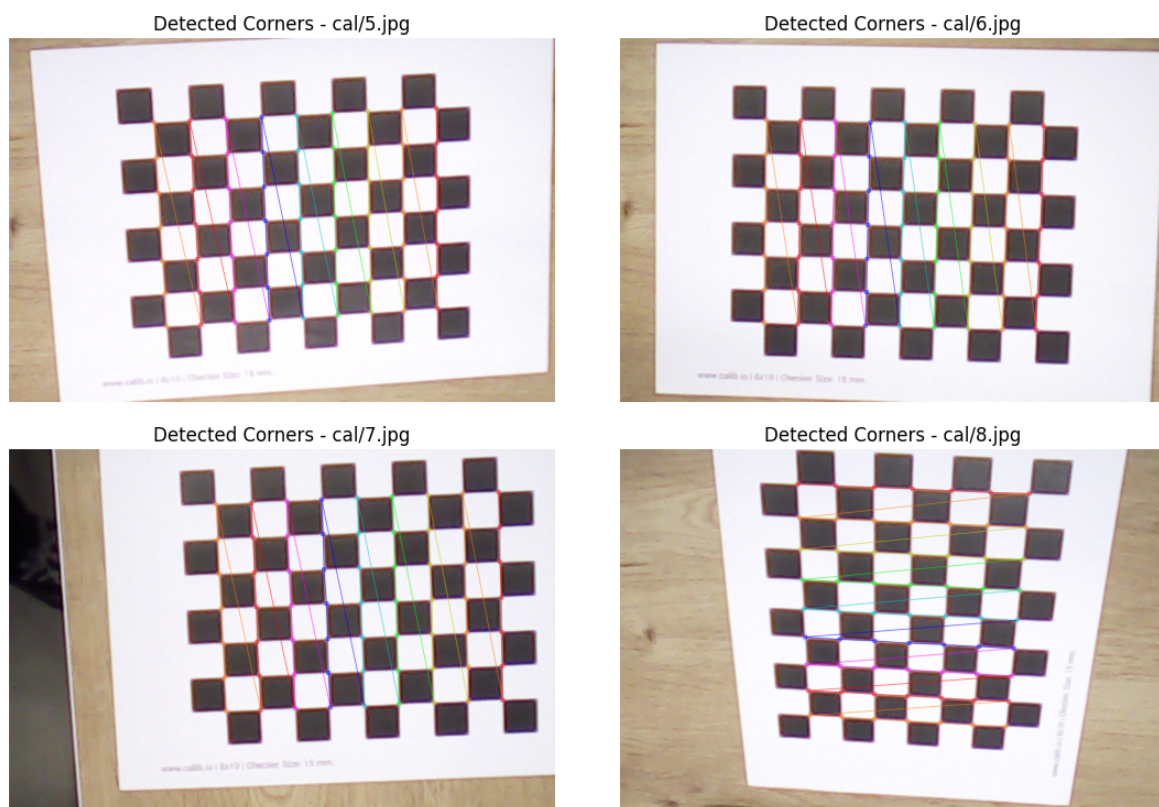


Figure 2: Detected Corner Points

4.4 Intrinsic Parameters, Distortion Coefficients and Reprojection Error

4.4.1 Stereo Setup from Single Camera

The intrinsic calibration results indicate the internal parameters of a single camera. The intrinsic matrix (K) shows that the camera has focal lengths of approximately 1555.94 pixels along the x-axis and 1561.49 pixels along the y-axis, with the principal point (optical center) located at (756.08, 497.56) in the image plane. The distortion coefficients reveal significant lens distortion, particularly radial distortion, with notable values such as -3.20 for the second coefficient and 15.80 for the fifth, suggesting strong barrel distortion. The reprojection error is 0.2654, which is relatively higher compared to ideal calibration, indicating a moderate level of accuracy in the parameter estimation. This result may benefit from more calibration images or improved corner detection to reduce error.

Calibration Parameters

Intrinsic Matrix (K):

$$\mathbf{K} = \begin{bmatrix} 1.55593613e+03 & 0.00000000e+00 & 7.56076066e+02 \\ 0.00000000e+00 & 1.56149035e+03 & 4.97557879e+02 \\ 0.00000000e+00 & 0.00000000e+00 & 1.00000000e+00 \end{bmatrix}$$

Distortion Coefficients:

$$[2.51602510e-01 \quad -3.20063051e+00 \quad 1.83156042e-02 \quad 1.08366488e-02 \quad 1.58031826e+01]$$

Reprojection Error: 0.26538355416644765

Calibration parameters including intrinsic matrix, distortion coefficients, and reprojection error.

4.4.2 Stereo Setup from Two Cameras

The intrinsic calibration results for the stereo camera setup reveal the internal parameters for both the left and right cameras. The left camera has focal lengths of approximately 1117.33 pixels (x-axis) and 1121.05 pixels (y-axis), with the principal point at (633.59, 352.86). Its distortion coefficients indicate moderate radial and minimal tangential distortion, with a reprojection error of 0.0411, signifying accurate calibration. The right camera shows focal lengths of 1127.91 pixels (x) and 1129.31 pixels (y), and a principal point located at (662.00, 362.09). Its distortion is slightly less pronounced, and the reprojection error is 0.0418, also indicating high calibration accuracy. These results confirm that both cameras are well-calibrated and suitable for reliable stereo vision tasks.

Left Camera Calibration Parameters

Camera Matrix1 (K1):

$$\begin{bmatrix} 1.11733489e+03 & 0.00000000e+00 & 6.33591415e+02 \\ 0.00000000e+00 & 1.12105071e+03 & 3.52857415e+02 \\ 0.00000000e+00 & 0.00000000e+00 & 1.00000000e+00 \end{bmatrix}$$

Distortion Coefficients1:

$$\begin{bmatrix} 1.49600973e-01, & -1.16724498e+00, & 2.06769613e-05, \\ & 1.17632493e-03, & 1.91543658e+00 \end{bmatrix}$$

Reprojection Error1: 0.04115133280552776

Right Camera Calibration Parameters

Camera Matrix (K2):

$$\begin{bmatrix} 1.12791070e+03 & 0.00000000e+00 & 6.62008487e+02 \\ 0.00000000e+00 & 1.12930803e+03 & 3.62094885e+02 \\ 0.00000000e+00 & 0.00000000e+00 & 1.00000000e+00 \end{bmatrix}$$

Distortion Coefficients2:

$$\begin{bmatrix} 0.11197568, & -0.82377783, & -0.00251442, \\ & 0.00395234, & 0.93550175 \end{bmatrix}$$

Reprojection Error2: 0.041811447380338776

Left and Right Camera Calibration Parameters

After calibrating the cameras individually, we pass the intrinsic parameters and the distortion coefficients to the OpenCV function `stereo.Calibrate()` function which returns the extrinsic parameters rotation and translation between the two cameras and also returns the fundamental and essential matrix.

As the intrinsics of the both cameras were calculated before, we used flags for the fixing the intrinsic parameters and computing the extrinsics and epipolar geometry from the `stereo.Calibrate()` function.

Results of Stereo Calibration

$$R = \begin{bmatrix} 0.9955 & 0.0331 & 0.0883 \\ -0.0317 & 0.9994 & -0.0167 \\ -0.0888 & 0.0139 & 0.9960 \end{bmatrix} \quad T = \begin{bmatrix} 7.8245 \\ 0.5082 \\ 3.0859 \end{bmatrix}$$

$$F = \begin{bmatrix} 0.00000315 & -0.00018282 & 0.0997 \\ 0.00022430 & -0.00000037 & -0.6423 \\ -0.1340 & 0.6441 & 1 \end{bmatrix} \quad E = \begin{bmatrix} 0.0528 & -3.0769 & 0.5578 \\ 3.7671 & -0.0063 & -7.5203 \\ -0.7542 & 7.8027 & -0.1758 \end{bmatrix}$$

Results from Stereo Calibration

4.5 Challenges

During the process of camera calibrations following problems were faced:

- Reprojection error was high when the checkerboard pictures were placed at improper orientation.
- For reducing the reprojection error, we increased the number of images and captured the checkerboard images placed at different angles, making sure no noise is present while capturing the image and corners are identified in each image.
- Minimum threshold of reprojection error was set to 0.5. We recalibrated the cameras until we get the values below 0.5 and were able to reduce below 0.1 pixels.

5 Image Undistortion

The left and right stereo images (`1.jpg` and `2.jpg`) were loaded and converted to grayscale for processing. Using the precomputed camera matrix **K** and distortion coefficients **dist** from calibration, the images were undistorted via OpenCV's `cv2.undistort()` function. This correction removes lens-induced radial and tangential distortions, ensuring geometric accuracy for subsequent stereo processing. The results were visualized using Matplotlib, with both undistorted images displayed side-by-side for qualitative verification. The undistorted images show improved straight-line preservation compared to the originals, confirming effective distortion correction. This preprocessing step is critical for accurate feature matching and stereo depth computation.

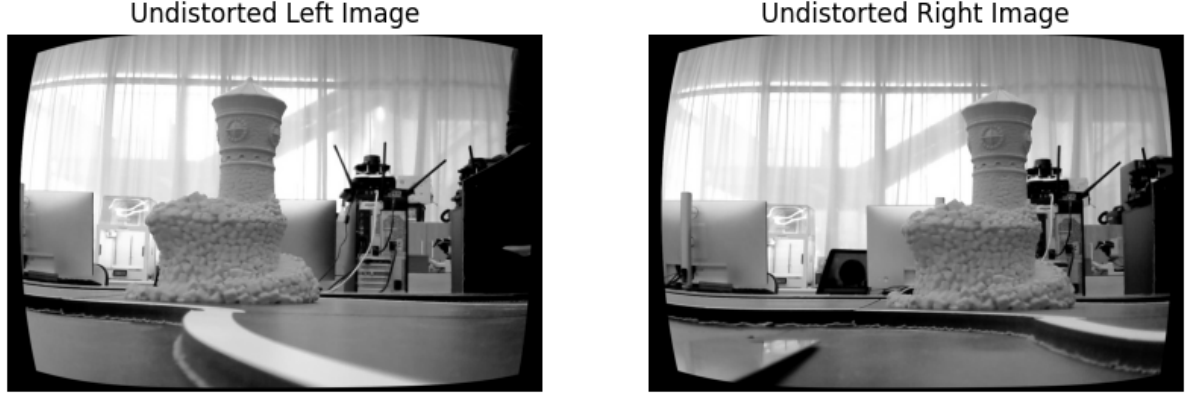


Figure 3: undistorted (left) and (right) images

6 Image Rectification

In image rectification, a critical step in 3D vision systems, the views from two cameras—left and right—are aligned onto a common image plane. Rectification ensures that corresponding points between the stereo images lie on the same horizontal line, simplifying the process of depth estimation. The function `rectify_images` takes the original stereo images (`img1` and `img2`), their respective intrinsic matrices (K , $K1$), and distortion coefficients (`dist`, `dist1`), along with an estimated transformation matrix (T) encoding the relative pose between the cameras.

Using OpenCV's `stereoRectify` function, the code computes the rectification transforms ($R1$, $R2$), projection matrices ($P1$, $P2$), and a disparity-to-depth mapping matrix (Q). The `initUndistortRectifyMap` and `remap` functions then apply these transformations to produce rectified images.

The output images, `rectified_left` and `rectified_right`, are saved and displayed, showing corrected views suitable for stereo matching.

Below is a visual representation of the rectified left image and right image.

6.1 Single Camera Setup

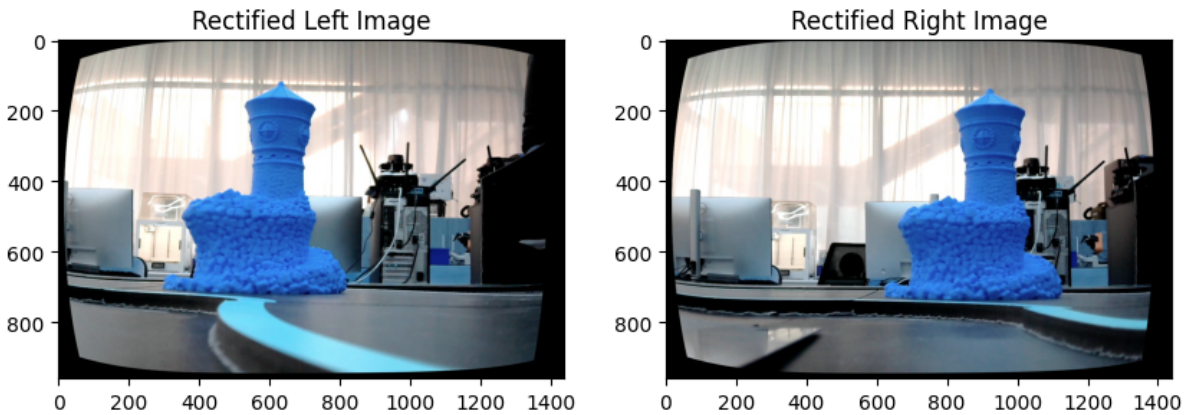


Figure 4: Rectification result for a single camera.

6.2 Dual-Camera Setup

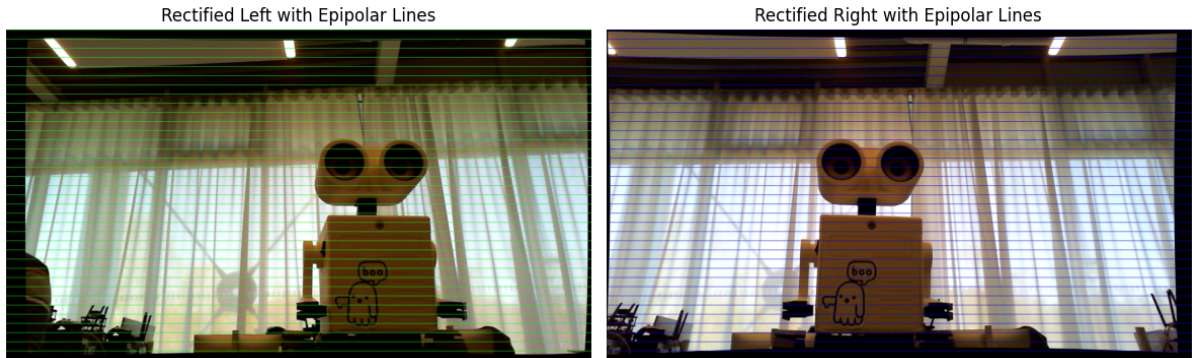


Figure 5: Rectification result for a Dual camera.

6.3 Challenges

For rectifications following problems were faced:

- Rectification will only be correct if the intrinsic parameters and the lens distortions of both cameras are calculated accurately. We undistorted the images removing the lens distortions to properly align the images.
- Slight misalignments were observed in the rectified image pairs obtained from the dual-camera setup. This was primarily due to the two cameras having non-identical distortion coefficients, introducing ambiguity in the geometric alignment and affecting the accuracy of stereo rectification.
- Cameras were aligned properly along the straight line to properly rectify the images. Misalignment led us to wrong rectifications.
- While capturing stereo image pairs, object should be placed such that all the regions of the object are visible in the both images to avoid partial occlusions. These occlusions leads to improper feature matching

7 Sparse 3D Reconstruction

7.1 Feature Matching

The feature matching implementation involves detecting distinctive keypoints in two rectified stereo images, computing their descriptors, and then finding correspondences between them using a Brute-Force Matcher (BFMatcher) with Hamming distance for binary descriptors. Specifically, the AKAZE (Accelerated-KAZE) algorithm is used for feature detection and description, which is robust against scale, rotation, and noise variations. After detecting keypoints ($kp1$, $kp2$) and computing their descriptors ($des1$, $des2$), the BFMatcher performs an exhaustive search to find the best matches by comparing each descriptor in the first image with all descriptors in the second image. The matches are then sorted by their distance scores, where a lower distance indicates a stronger match.

To improve matching accuracy, a distance threshold ($threshold = 100$) is applied to filter out weak matches, retaining only the most reliable correspondences.

7.1.1 Single Camera Setup

For single camera approach, in total we get 2368 matches, and excluding the outliers, we get total of 2108 matches.

Results can be visualized below:



Figure 6: Feature Matching from single camera approach.

7.1.2 Dual Camera Setup

For dual camera approach, in total we get 400 matches, and excluding the outliers, we get total of 369 matches.

Results can be visualized below:

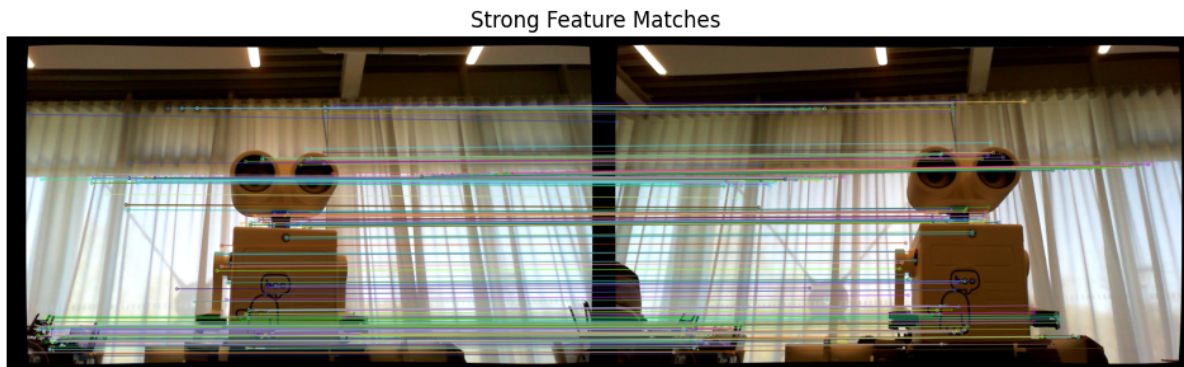


Figure 7: Feature Matching from dual camera approach.

7.2 Challenges

- SIFT was implemented initially for feature detection with FLANN based matcher, with Lowe's Ratio test with threshold of 0.75. It computed only good matches of 254. By changing different parameters, more good points were achieved but mismatches were increased.
- With ORB algorithm for features extraction with BFMatcher, good matches included the more outliers and mismatches.
- AKAZE algorithm for feature detection with BFMatcher was able to extract most correct points among stereo images, as it has binary descriptor for the feature matching. It performed better with repetitive background and gave robust results
- Feature matching was difficult in our case due to presence of white light in the background, main object had minor occlusions, right side of the object is not fully visible in the right image as compared to the left image.

7.3 Finding Fundamental Matrix

Fundamental matrix is computed using the RANSAC algorithm which filter outs the outliers from the matched feature points. We have also tested an 8-point algorithm which takes at least 8 matched feature points but does not filter the incorrect points from the matched points. RANSAC is more robust as compared to the 8 point algorithm.

Correctness of the fundamental matrix is checked by checking the rank of the matrix which should be equal to 2. Also epipolar constraints are checked if they satisfy the condition:

$$\mathbf{x}'^T F \mathbf{x} = 0$$

This equation expresses the epipolar constraint, where $\mathbf{x}$ and $\mathbf{x}'$ are corresponding image points in homogeneous coordinates, and F is the fundamental matrix describing the epipolar geometry between two views.

7.3.1 Single Camera Setup

The results below show us the fundamental matrix and essential matrix for our single camera setup that we used for stereo vision. The results indicate if our calibration and rectification are accurate, hence the values in the results.

Dual Camera

Fundamental Matrix (F):

$$\begin{bmatrix} -5.7828e-23 & -5.9257e-06 & 3.1942e-04 \\ 5.9257e-06 & -5.8648e-22 & -7.4436e-03 \\ -3.1942e-04 & 7.4436e-03 & 0.0000e+00 \end{bmatrix}$$

Rank of F: 2

Single Camera Fundamental Matrix and Rank

7.3.2 Dual Camera Setup

These are the results we get from using two cameras in stereo vision the results that are shown below. The results define our camera calibration and stereo rectification.

Dual Camera

Fundamental Matrix (F):

$$\begin{bmatrix} 2.42486708e-08 & -1.98373359e-06 & 1.73949927e-03 \\ 4.01892419e-06 & -1.75360384e-06 & 2.81065944e-02 \\ -3.22242331e-03 & -2.81755180e-02 & 1.00000000e+00 \end{bmatrix}$$

Rank of F: 2

Dual Camera Fundamental Matrix and Rank

Epipolar Constraints

Epipolar constraint satisfied for point pair 0: -0.001 ... Epipolar constraint satisfied for point pair 36: 0.004 ... Epipolar constraint satisfied for point pair 108: 0.003 ...

Epipolar Constraints

7.4 Extracting Essential Matrix and Camera Pose

Fundamental matrix is transformed into the essential matrix using the camera intrinsic parameters following the equation:

$$\mathbf{E} = \mathbf{K}^\top \mathbf{F} \mathbf{K}$$

Singular Value Decomposition (SVD) is applied on the computed essential matrix to calculate the relative rotation matrices of the two cameras and the translation between the two cameras.

7.4.1 Single Camera Setup

Essential Matrix

$$\begin{bmatrix} -1.4000e-16 & -1.4397e+01 & -4.0905e+00 \\ 1.4397e+01 & -1.4300e-15 & -4.6271e+00 \\ 4.0905e+00 & 4.6271e+00 & 4.4409e-16 \end{bmatrix}$$

Rotation Matrices

Rotation Matrix (R1):

$$\begin{bmatrix} 1.0000e+00 & 5.3939e-18 & -9.7326e-17 \\ -4.9501e-17 & 1.0000e+00 & -2.2449e-17 \\ -2.0385e-16 & 5.3062e-18 & 1.0000e+00 \end{bmatrix}$$

Rotation Matrix (R2):

$$\begin{bmatrix} -0.8255 & -0.1542 & 0.5429 \\ -0.1542 & -0.8636 & -0.4799 \\ 0.5429 & -0.4799 & 0.6892 \end{bmatrix}$$

Rotation matrices (R1 and R2) obtained by SVD decomposition of essential matrix.

Rotation Matrices (R1 R2)

The rotation matrix R1 is nearly an identity matrix, with all diagonal values very close to 1 and off-diagonal elements on the order of 10 to power -16, indicating negligible rotation. This suggests that the camera orientation was nearly unchanged between the two shots, which aligns with your intention of pure translation for synthetic stereo.

In contrast, R2 shows significant rotation. The values indicate a rotation of roughly 45 degrees around the principal axes. This unexpected transformation implies that during the motion or between captures, the camera experienced an unintended rotation, contradicting the assumption of pure translational movement.

Translation Vector

$$[0.99124537 \quad -0.04256718 \quad -0.06589214]$$

Translation vector between the two camera frames.

The translation vector confirms that the dominant motion is along the X-axis, as expected when physically shifting the camera horizontally to emulate a stereo baseline. The small values in Y and Z may result from hand-induced jitter or minor misalignment during the manual shift.

The direction and scale of the translation vector are consistent with stereo geometry assumptions. However, the combination with the rotation in R2 reduces accuracy in depth recovery unless corrected or

refined via bundle adjustment.

7.4.2 Dual Camera Setup

Essential Matrix

$$\begin{bmatrix} 3.02729438e-02 & -2.48480300e+00 & 1.17866307e+00 \\ 5.03406049e+00 & -2.20384984e+00 & 3.36698367e+01 \\ -1.99885888e+00 & -3.36888817e+01 & 2.82510189e-01 \end{bmatrix}$$

Computed Essential Matrix using intrinsic parameters and fundamental matrix.

Rotation Matrices

Rotation Matrix (R1):

$$\begin{bmatrix} -0.97069742 & 0.09150394 & 0.2222016 \\ 0.09769263 & 0.99507146 & 0.01699821 \\ 0.21955107 & -0.03820758 & 0.97485256 \end{bmatrix}$$

Rotation Matrix (R2):

$$\begin{bmatrix} -0.99643162 & 0.02774334 & 0.07971402 \\ -0.03001058 & -0.99917434 & -0.02738607 \\ -0.07888843 & 0.02968062 & -0.99644151 \end{bmatrix}$$

Rotation matrices (R1 and R2) obtained by SVD decomposition of essential matrix.

Rotation Matrices (R1 R2) The rotation matrices you obtained indicate that both the left and right cameras are noticeably rotated relative to the rectified stereo coordinate system. The values in R1 and R2 show significant deviations from the identity matrix, with nonzero off-diagonal elements suggesting combined yaw, pitch, and roll misalignment. This means that the cameras are not perfectly fronto-parallel, and rectification had to apply substantial rotational corrections.

This result is valid in the mathematical sense, but it may not reflect the intended physical configuration if the cameras were assumed to be nearly parallel. Such large rotations could be caused by inaccuracies in corner detection during calibration, poor initial intrinsic parameters, or a slight misalignment in how the cameras were mounted.

Translation Vector

$$[0.99686629 \quad -0.03429869 \quad -0.07128248]$$

Translation vector between the two camera frames.

Translation Vector (T) Your translation vector indicates that the right camera is located almost entirely along the X-axis from the left camera, with a small downward and backward shift. This is consistent with a typical stereo setup where cameras are side-by-side and mostly horizontally aligned.

The result is physically reasonable and suggests a dominant horizontal baseline, which is ideal for depth estimation. The slight deviations in Y and Z may result from minor mechanical misalignment or noise in the calibration data.

To check the computation we have applied the following checks:

1. Essential matrix determinant should be equal to zero and has rank 2:

Essential Matrix Properties

Rank of $\mathbf{E}$: 2

Essential matrix satisfies $\det(\mathbf{E}) = 0$ condition.

2. Determinant of the rotation matrices should be equivalent to 1 as there was no relative rotation in between the cameras:

Determinant of Rotation Matrices

Det($\mathbf{R1}$): 0.9999999999999999

Det($\mathbf{R2}$): 0.9999999999999997

7.5 Triangulation

For triangulation, we utilize OpenCV's `cv2.triangulatePoints()` function. We construct the projection matrices for both cameras, designating the first camera as the reference frame. After preparing the 2D point correspondences in homogeneous coordinates, we perform triangulation to recover the 3D scene points.

To validate the results, we reproject the 3D points onto the original image:

7.5.1 Single Camera Setup

In the single camera setup, the image shows the original scene overlaid with green dots representing the re-projected 3D points obtained through triangulation. These 3D points are reconstructed by matching corresponding 2D points across frames and using the projection matrix of the single camera. The re-projected points appear densely around the central object (an orange figurine), outlining its structure and surroundings. This demonstrates the recovery of 3D geometry from 2D correspondences, and the reprojected points visually align with meaningful edges and textures in the image.

Evaluation:

The result appears moderately accurate but not perfect. While many reprojected points lie on the object's surface and surrounding structures, there are noticeable outliers and some noise, especially around the borders. This could be due to poor feature matching, insufficient parallax (baseline) in the single camera setup, or errors in pose estimation between frames. Since a single camera relies on motion to generate disparity, any slight calibration error, lack of distinctive features, or inaccurate matching can degrade the triangulation quality.

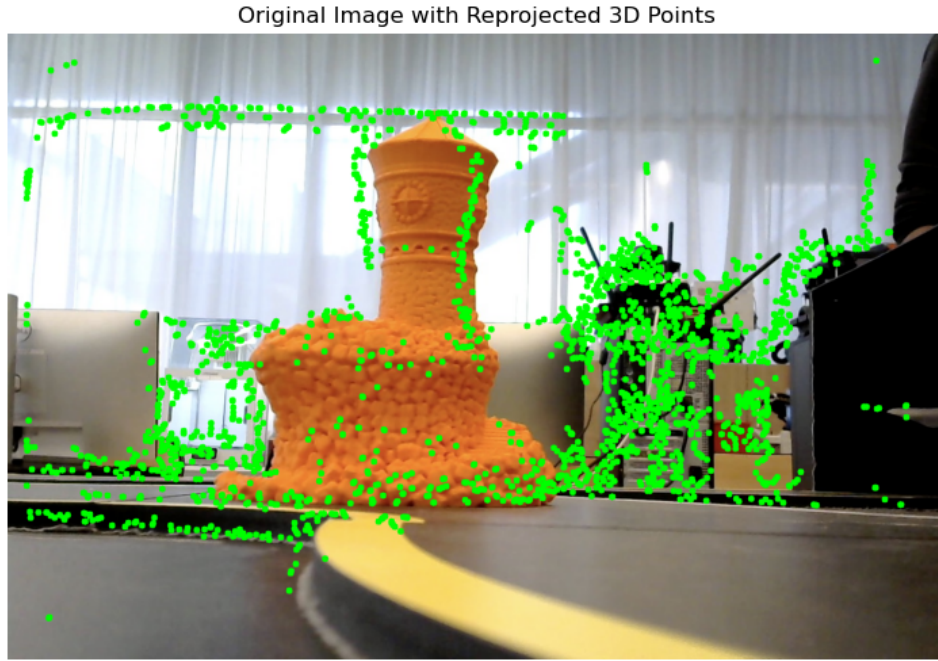


Figure 8: Reprojected 3D points onto the original image

7.5.2 Dual Camera Setup

The dual camera setup shows a similar reprojection visualization, with green points superimposed on the original image. Here, stereo triangulation is used, where corresponding features in synchronized left and right images are matched and triangulated using both cameras projection matrices. The resulting 3D points appear mainly concentrated on the robot figure and background elements like curtains. This suggests that stereo vision was able to recover depth across multiple planes in the scene.

Evaluation:

The result is somewhat sparse and exhibits noticeable noise, particularly above and below the robot and in less-textured regions. Although stereo systems theoretically provide more accurate triangulation due to known baseline and simultaneous capture, the quality here seems limited. Potential causes include poor stereo calibration, suboptimal rectification, or inadequate feature matching due to repetitive patterns (e.g., curtains) or reflections.

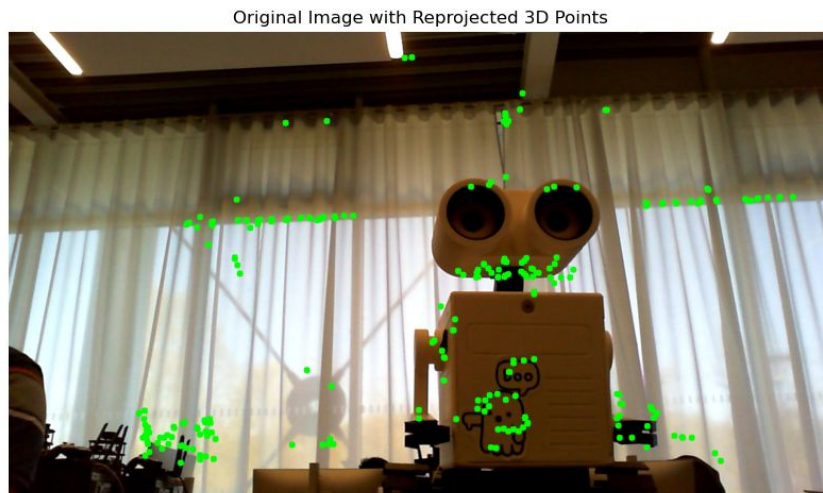


Figure 9: Reprojected 3D points onto the original image

8 Depth MAP from Stereo Images

8.1 The Semi-Global Block Matching (SGBM)

Depth map of the images is computed after the rectification of the images using the OpenCV Semi-Global Block Matching (SGBM) algorithm. For disparity estimation, OpenCV's StereoSGBM method is used to construct a stereo matcher. The algorithm balances local accuracy with global smoothness constraints, making it suitable for moderately complex depth estimation tasks.

The key parameters configured include `minDisparity`, which defines the minimum possible disparity which is set to zero as the images are properly rectified, `numDisparities` value of 64, which determines the search range for corresponding pixels and it should be a multiple of 16 and a `blockSize` of 11, which affects the size of the comparison window. Penalty terms `P1` and `P2` were scaled with the square of the block size to enforce disparity smoothness, while parameters like `uniquenessRatio` and `speckleWindowSize` help reduce mismatches and speckle noise.

The `disp12MaxDiff` parameter ensures left-right consistency by rejecting disparity values that differ significantly when calculated from left-to-right versus right-to-left images. Finally, the `preFilterCap` parameter helps to normalize image brightness before matching by clipping pixel values, which can improve robustness in varying lighting conditions. All these parameters collectively contribute to the generation of a reliable and accurate disparity map, forming the basis for depth estimation and 3D reconstruction.

8.2 Disparity Map Computation

Disparity maps are computed using the stereo left and stereo right matches, which estimate pixel shifts between the left and right rectified images. The left disparity map is generated with `stereo-left.compute(left-img, right-img)`, while the right disparity map is obtained with `stereo-right.compute(right-img, left-img)`.

These values are initially in fixed-point format, so they are converted to floating-point by dividing by 16.0 for accurate depth representation.

The disparity maps indicate horizontal displacement, where larger disparities correspond to closer objects and smaller disparities to those further away. Computing disparity from both perspectives enables later filtering to improve depth accuracy and reduce inconsistencies.

The initial disparity map contains noise and artifacts, especially around depth discontinuities and textureless regions. To address this, a Weighted Least Squares (WLS) filter is applied to the left disparity map. This filter takes into account the structure of the image, preserving edges while smoothing homogeneous areas. The `lambda` parameter was set to 8000 to control the degree of smoothing, and `sigmaColor` was set to 2.5 to ensure that edges in the image are respected. The filtered disparity is then normalized for visualization, allowing for a clear and interpretable disparity map display.

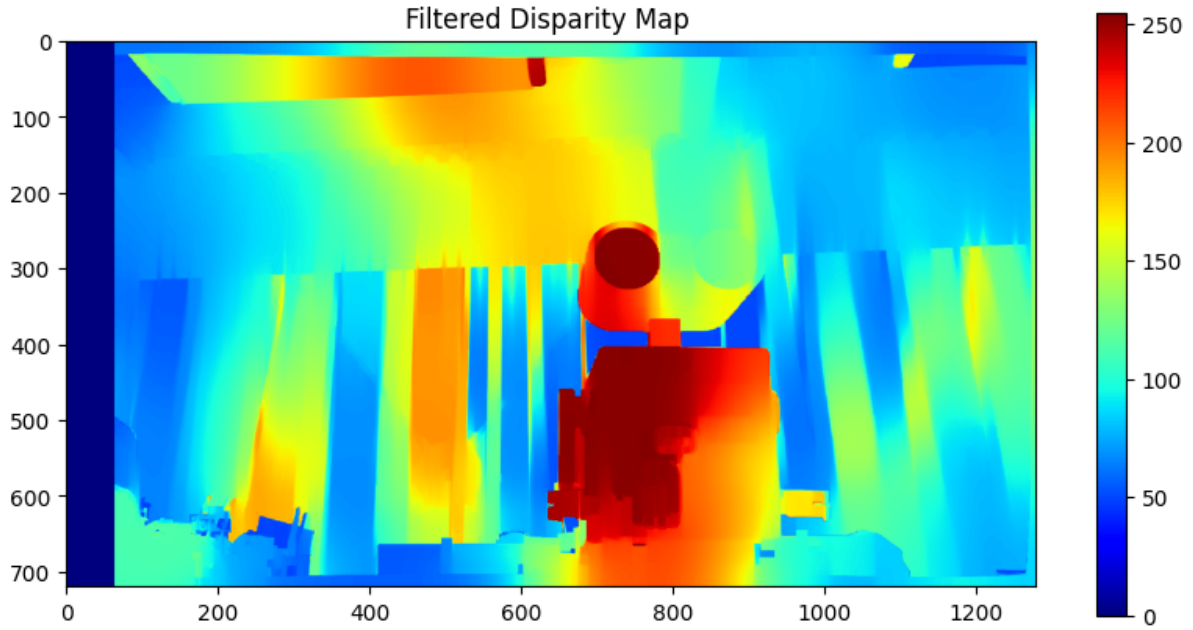


Figure 10: Disparity map

As the object was placed near to the camera, red color displays higher disparity values and the regions further away from the camera are displayed by blue color.

After the disparity map is filtered, it is reprojected into 3D space using the stereo camera's reprojection matrix Q which we obtain from stereo rectification. The `cv2.reprojectImageTo3D` function performs this mapping, producing a 3D coordinate for each pixel in the disparity map. However, not all 3D points are reliable; therefore, a validity mask is used to filter out points with disparity values below a threshold and those containing invalid (NaN or infinite) coordinates. This step ensures the integrity of the resulting 3D point set. 3D point cloud obtained is shown below.

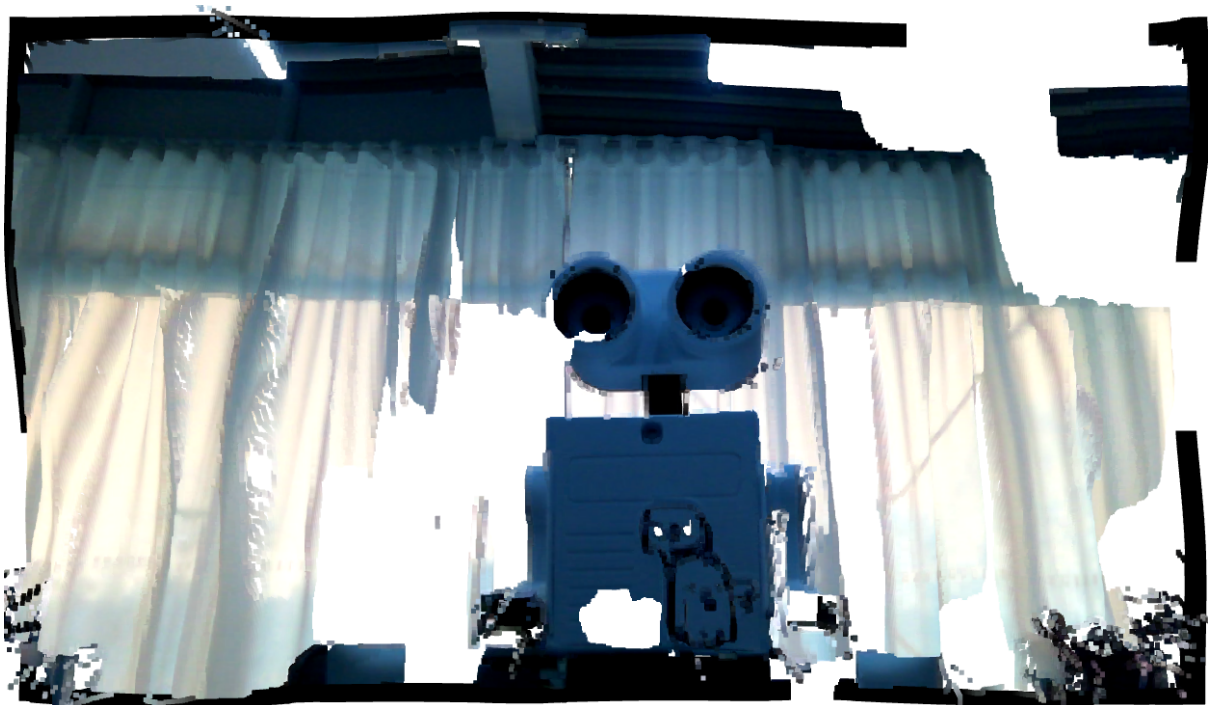


Figure 11: Disparity map

8.3 Challenges

Mismatching can be observed in the disparity map, this is due to the presence of occlusions in the rectified image. Upon careful observation we found that object placed is white whereas the background color is also white, this is the reason we can see some warmer colors in the background as well which should be displayed in cooler colors. Also we can see right side of the object is not properly remapped, this is because the left and right images had occlusions of the object's right side which was not completely visible. Some minor artifacts or vertical streaks may be observed in low-texture regions (e.g., walls), which are inherent limitations of block-matching algorithms in the absence of strong image gradients. Results can be improved if we take stereo images with good lighting conditions, choosing the background distinct with the other objects present in the scene to avoid occlusions and mismatches and also make sure the view angle for the scene of both angles is same, capturing all the scene information from both views.

9 Graphical User Interface

To make the project user friendly we used a simple GUI to run the code and perform 3D projection on two images taken by two cameras. In the GUI the tasks are performed, starting from calibrating the cameras by uploading checkerboard pictures till generating the point cloud in dense or sparse reconstruction.

9.1 Calibration

First step is to get parameters through calibrating both cameras, for that we give pictures of checkerboard to the GUI through the buttons as shown in the figure below.

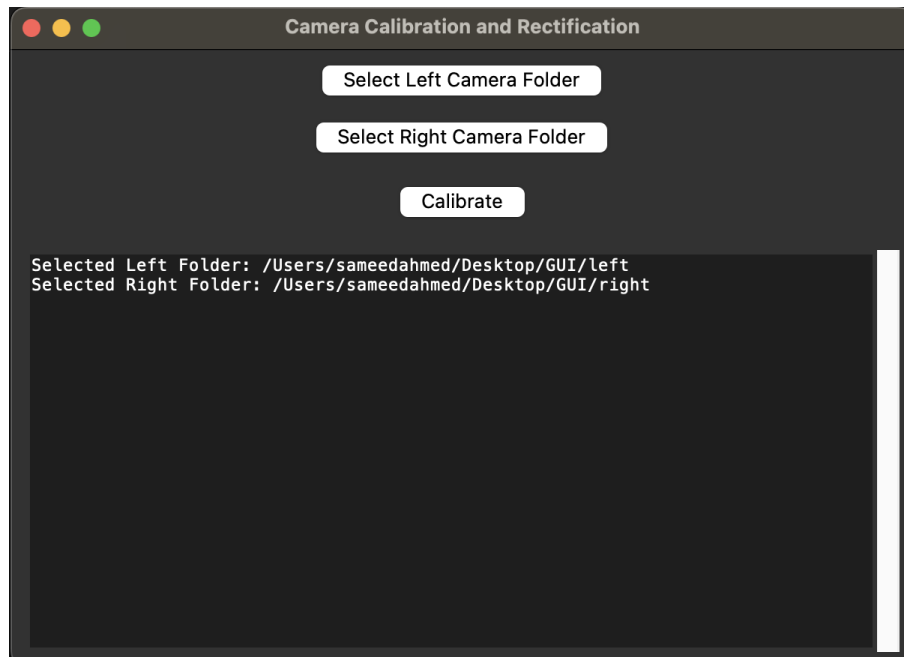


Figure 12: GUI home page

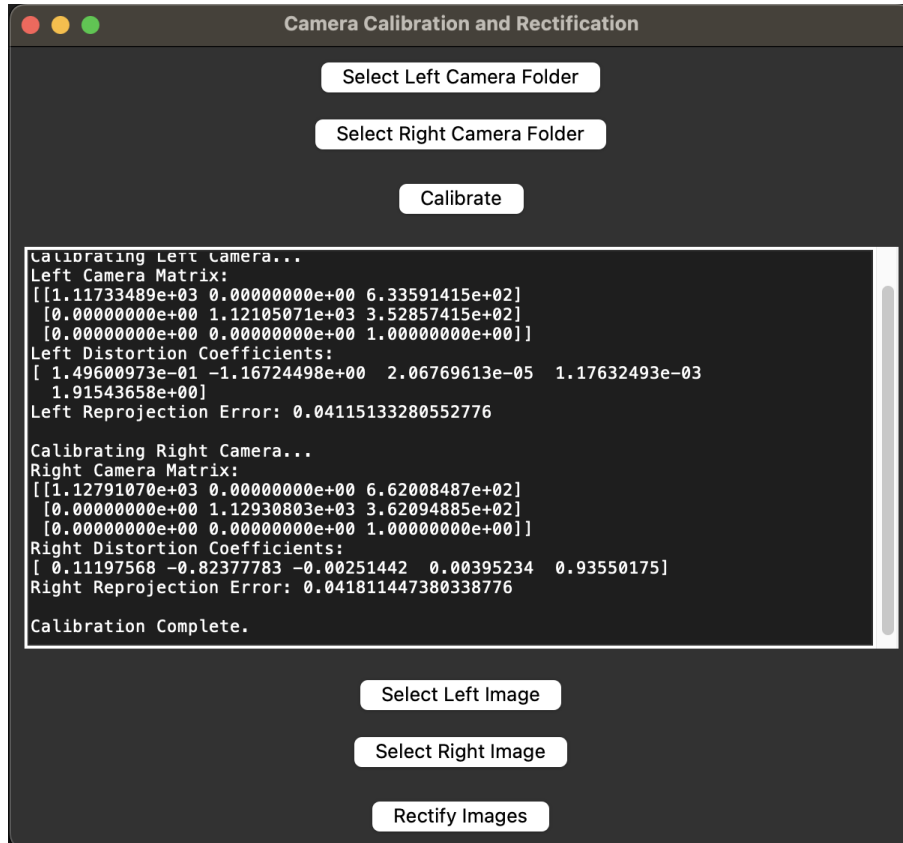


Figure 13: camera parameters attained through calibration

The figure show that the calibration is complete and the results of the calibration that are the camera matrix and the reprojection error are shown.

9.2 Rectification

After the cameras are calibrated, results can be seen in the log. Next step is to select the image pairs of the scene taken form both cameras and rectify them.

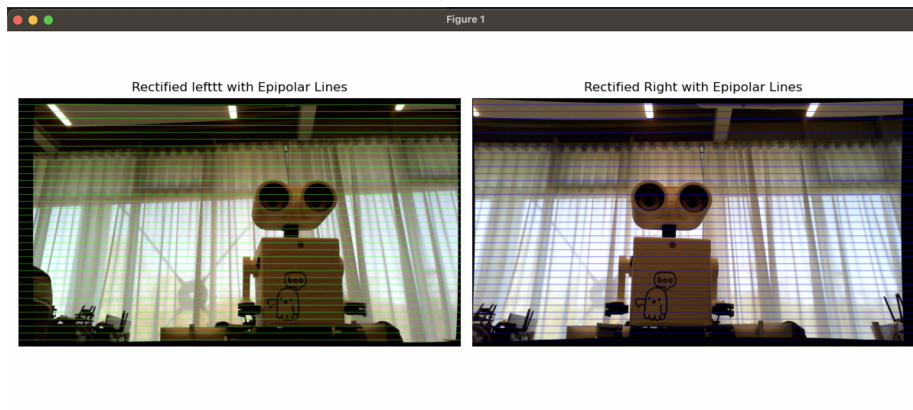


Figure 14: Rectified left and right image

After the stereo images are rectified, rectified pairs are displayed as in figure 14 and Q matrix is printed in the log.

9.3 3D Reconstruction

After image rectification we get two options to select:

- Sparse reconstruction
- Dense reconstruction

As shown in the figure 15, the options are shown clearly.



Figure 15: Selection for 3D reconstruction

9.3.1 Sparse

By Selecting sparse, we get the result of strong feature matches and reprojected points on the original image, shown in figure 16 and figure 1, respectively. The results like fundamental matrix, essential matrix, relative rotation and translation matrix are shown in the log with their checks. These results in the log are the parameters used in 3D reconstruction, figure 17 clearly shows the parameters in the log.

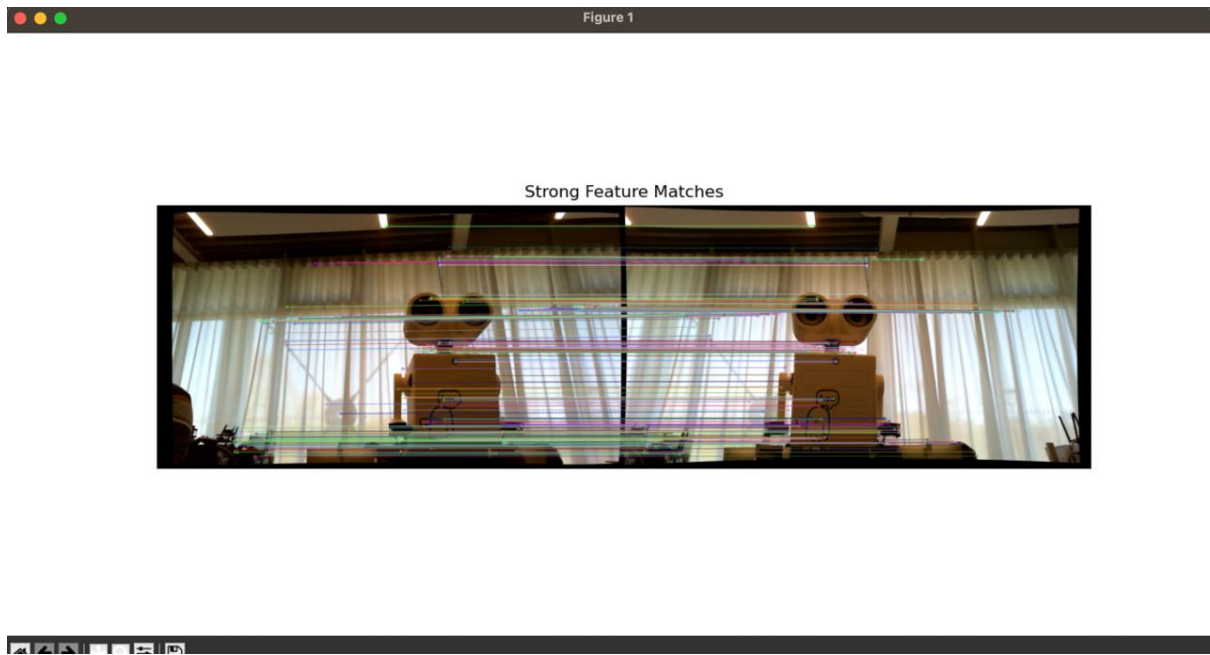


Figure 16: Matched features in rectified images

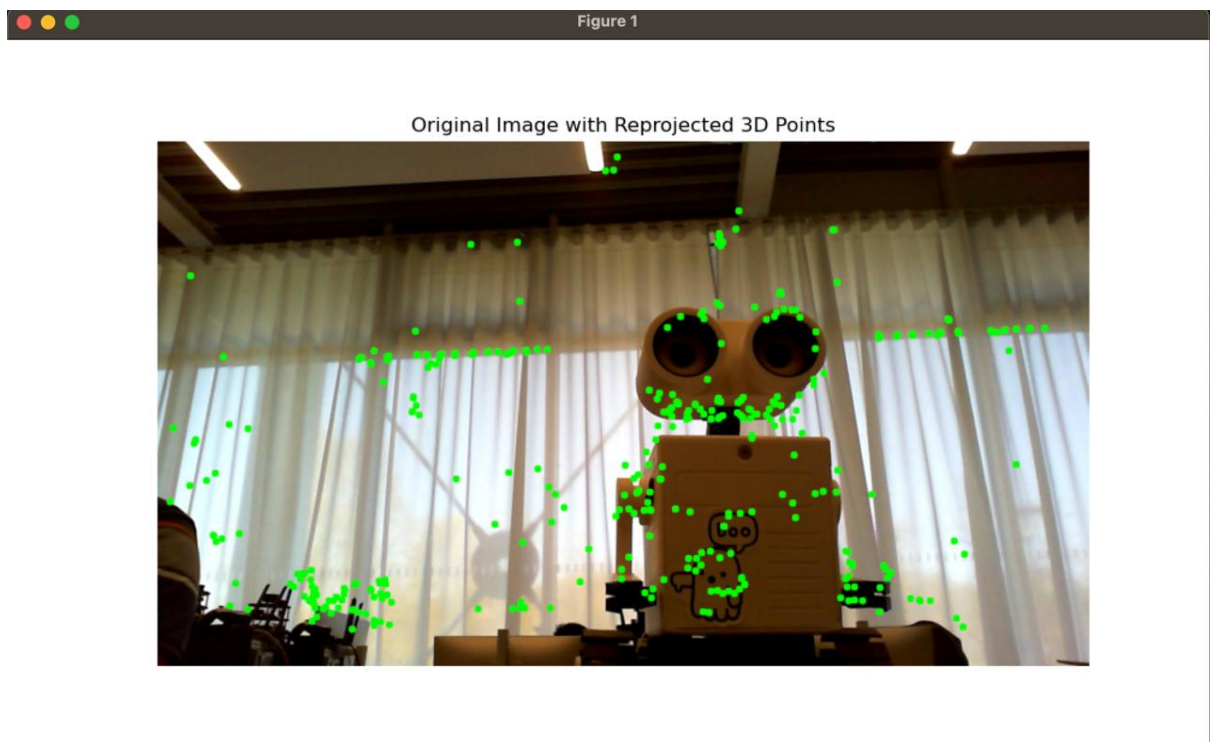


Figure 17: Reprojection of the points onto the image



Figure 18: Log showing parameters attained through triangulation

9.3.2 Dense

If we select the dense button, stereo SGBM algorithm is run on the selected two stereo image pairs and in its result a disparity map is visualized as an output, shown in figure 19. Along with the disparity map, a point cloud is also generated from the disparity as an output.

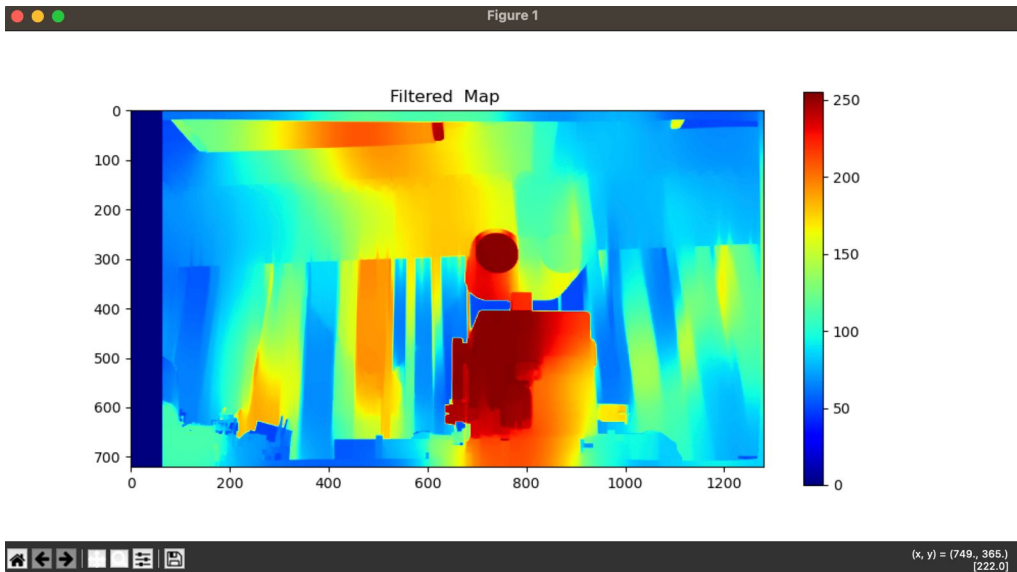


Figure 19: Disparity Map

10 Future Improvements

To enhance the system, future work could integrate deep learning-based feature descriptors (e.g., Super-Point) to improve matching robustness in low-texture regions. Implementing a post-processing pipeline with conditional random fields (CRFs) or multi-view consistency checks could refine disparity maps and reduce noise. Real-time performance could be achieved by optimizing SGBM parameters or leveraging GPU acceleration. Additionally, adopting structure-from-motion (SFM) techniques for the single-camera setup would mitigate rotational errors during motion. Expanding the GUI to support video input, dynamic parameter tuning, and interactive 3D visualization would improve user engagement. Finally, exploring active stereo methods with structured light could address occlusions and texture limitations in passive stereo systems.

11 Conclusion

In this project we have learned and implemented the basic steps for 3D reconstruction from only 2 images using sparse and dense approaches using OpenCV. In case of sparse approach, number of features extracted can be increased by more images of the same scene at different angles. Results can be further improved by using high quality stereo cameras.

By leveraging OpenCV's calibration tools, the system achieved accurate intrinsic and extrinsic parameter estimation, with stereo setups yielding reprojection errors below 0.1 pixels. The rectification process aligned epipolar geometry effectively, simplifying feature correspondence and enabling disparity computation. Sparse reconstruction, powered by AKAZE feature detection and RANSAC-based outlier rejection, generated 3D point clouds through triangulation, though limitations in parallax and occlusions introduced noise, particularly in the single-camera approach. For dense reconstruction, the Semi-Global Block Matching (SGBM) algorithm produced disparity maps, but performance suffered in textureless regions and under inconsistent lighting.

The single-camera stereo setup demonstrated how structure-from-motion techniques can effectively recover 3D structure using only one moving camera, while the two-camera stereo system allowed for simultaneous capture and provided more stable and accurate depth estimation due to known baseline and calibration. The inclusion of a GUI streamlined workflows, enabling users to calibrate cameras, rectify images, and reconstruct scenes with minimal manual intervention. Challenges such as lens distortion ($k_1=15.80$), stereo misalignment, and disparity noise underscored the trade-offs between algorithmic simplicity and real-world applicability. Overall, this project highlighted the strengths and limitations of each method and provided valuable insight into practical 3D reconstruction techniques using standard image inputs.

12 References

1. OpenCV Calibration Tutorial.
https://docs.opencv.org/4.x/d9/db7/tutorial_py_table_of_contents_calib3d.html
2. OpenCV Calibration and 3D Reconstruction Module.
https://docs.opencv.org/4.x/d9/d0c/group__calib3d.html
3. Stereo Vision – YouTube Playlist.
<https://youtube.com/playlist?list=PL2zRqk16wsdoCCLpou-dGo7QQNks1Ppzo&si=rAq-fQudIQCE2FDc>
4. A. Fusiello, *Computer Vision: Three-dimensional Reconstruction Techniques*. Springer, 2023.
5. Feature Matching in OpenCV.
<https://www.geeksforgeeks.org/feature-matching-in-opencv/>
6. Stereo SGBM Disparity Map Discussion.
<https://stackoverflow.com/questions/19951690/very-bad-disparity-map-using-stereo-sgbm/31501744>
7. Disparity Map Issues with Good Calibration.
<https://forum.opencv.org/t/getting-a-bad-disparity-map-with-good-calibration/4925/2>
8. Disparity Map Distance Calculation.
<https://answers.opencv.org/question/5846/calculating-distance-from-the-pixel-value-of-a-dispari>
9. D. Tiwari, “Introduction to Feature Detection and Matching.”
<https://medium.com/@deepanshut041/introduction-to-feature-detection-and-matching-65e27179885d>